

الجامعة الألمانية الأردنية
German Jordanian University

Control Systems II – Project

By:

Ramez Al-Masadeh

Ahmad Al-Amiri

Instructor: ***Dr.Mutaz Ryalat***

Contents

Executive Summary 3
Task & Summary of Achievements 3
MATLAP Scripts Analysis 4
Open loop system response 4
Controllability & Observability (Controllability.m) 5
Motive 5
Implementation 5
Pole Placement (poleplacemnt.m) 5
Motive: 5
Implementation: 5
Controller & Observer (test2.m) 7
1. Inverse Kinematics 7
2. Forward Kinematics Check 7
3. Linearized State-Space Model 8
4. Controllability & Observability 8
5. Pole Placement Controller 8
6. Precompensator (Nbar) 9
7. Closed-loop Simulation 9
8. Observer Design 10
9. Augmented Observer System 10
10. Workspace Visualization 11
LQR Control Analysis (LQR.m) 12
Implementation 12
Why it has Failed 12
Integral Control (Integral .m) 12
Motive: 12
Implementation: 12
AI Utilization 13
Appendix 1: MATLAB Commands Glossary 14
Appendix 2: References 17

Executive Summary

This report presents the design and analysis of a state-feedback controller with observer for the 5-bar linkage. The objective was to achieve precise end-effector positioning control with acceptable transient response characteristics. The system utilizes pole placement techniques for both controller and observer design, achieving successful tracking of desired end-effector positions.

Task & Summary of Achievements

The objective of this task is to design a state-space model for a 5-bar linkage robotic system. This includes performing a comprehensive analysis of the dynamic model to understand its structure and behavior. The system's stability and time-domain responses will be evaluated using analytical and simulation-based methods. Based on this analysis, an appropriate state-feedback controller will be designed to ensure the desired performance and system stability. Additionally, an observer will be developed to estimate the internal states of the system that are not directly measurable, enabling effective closed-loop control.

In this project, a control system was successfully developed and simulated for a 5-bar linkage robot. The key accomplishments include:

- System Modeling and Inverse Kinematics:

The forward and inverse kinematics were formulated to relate joint angles to end-effector positions. The desired Cartesian position was converted to corresponding joint angles using numerical methods.

- State-Space Linearization:

The nonlinear dynamics of the manipulator were linearized around the equilibrium point derived from the inverse kinematics, resulting in a simplified model suitable for control design.

- State-Feedback Control via Pole Placement:

A state-feedback controller was designed using pole placement to achieve desired dynamic performance (overshoot<10% and settling time<3s). A precompensator gain was also computed to eliminate steady-state error.

- Design and Implementation of a State Observer:

A Luenberger observer was added to estimate unmeasured state variables, enabling full-state feedback when only output measurements (end-effector position) are available.

- Simulation and Analysis:

Both the basic and observer-based closed-loop systems were simulated. Their performances were analyzed via step responses, with attention to overshoot, settling time, and tracking accuracy.

- LQR Controller Exploration:

In addition to pole placement, a Linear Quadratic Regulator (LQR) was tested for optimal state-feedback design. The LQR controller allowed trade-offs between control effort and performance to be tuned systematically via weighting matrices.

- Reachable Workspace Visualization:

The manipulator's physical reachability was mapped in Cartesian space. This helped to ensure that the desired end-effector positions were physically achievable within the system's joint limits.

MATLAP Scripts Analysis

Open loop system response

In this section I have tested the open loop systems response that proved the need for a controller, as shown in fig.1, open loop Response.

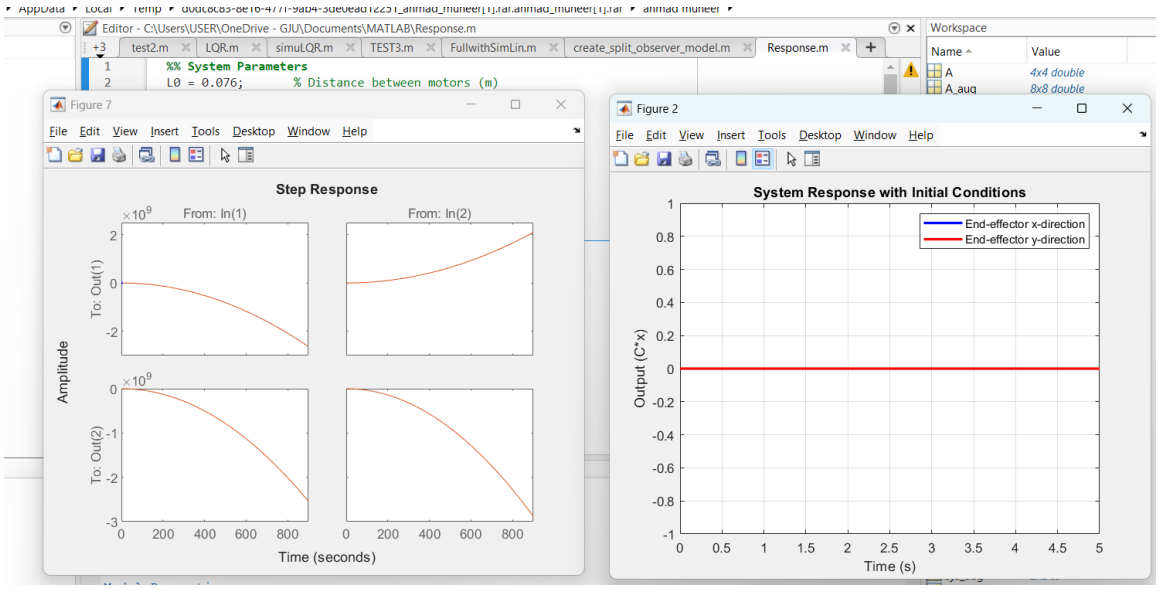


Fig.1

Controllability & Observability (Controllability.m)

Motive:

To ensure effective control and accurate state estimation, it is crucial to verify that the system is both controllable and observable.

Implementation:

The MATLAB script computes the controllability matrix using ctrb(A, B) and checks its rank against the number of system states (size(A,1)). A full rank indicates the system is controllable. Similarly, the observability matrix is computed using obsv(A, C), and its rank is evaluated. Full rank confirms observability. The code uses conditional checks to display whether the system satisfies each condition

Pole Placement (poleplacemnt.m)

Motive:

The goal of this section is to design a state-feedback controller that ensures fast and stable system performance. Specifically, we aim to achieve less than 10% overshoot and settling time under 5 seconds. To meet these time-domain specifications, we use pole placement to place the closed-loop poles at desired locations in the left-half of the complex plane, ensuring both speed and stability.

Implementation:

The code first defines the linearized state-space matrices ***A, B, C*** and ***D*** based on the dynamics of the 5-bar linkage robotic system. A set of desired pole locations is chosen to

shape the system response (poles = [-3 + 3j, -3 - 3j, -15, -16]), **where we choose the last 2 to be at least 5 times further to maintain the effect of the dominant poles.**

The place function is then used to compute the state-feedback gain matrix K such that the eigenvalues of the closed-loop system matrix Acl=A−BK match the desired poles. The resulting closed-loop system (sys_cl) is defined using the new state-space representation.

A step input is applied to the closed-loop system, and the responses for both output positions (X and Y) are recorded and plotted. The **stepinfo**() function evaluates the step response characteristics (e.g., rise time, settling time, and overshoot), helping confirm whether the performance goals have been met.

This code helped us to notice the following:

1. **Poor Performance with Pole Placement**
    - The initial pole placement controller resulted in large steady-state errors, failing to achieve the desired performance.

2. **Inaccuracy of step() Function**
    - The step() function was unsuitable for evaluation; **it only gives** a step input for **one of the inputs**. So ***lsim()*** was used to solve this issue.
3. **Invalid Linearization**
    - The linear model did not accurately capture the nonlinear mapping between joint angles and end-effector position, making it ineffective for precise control.

As shown in fig.2 for the Pole Placement response.

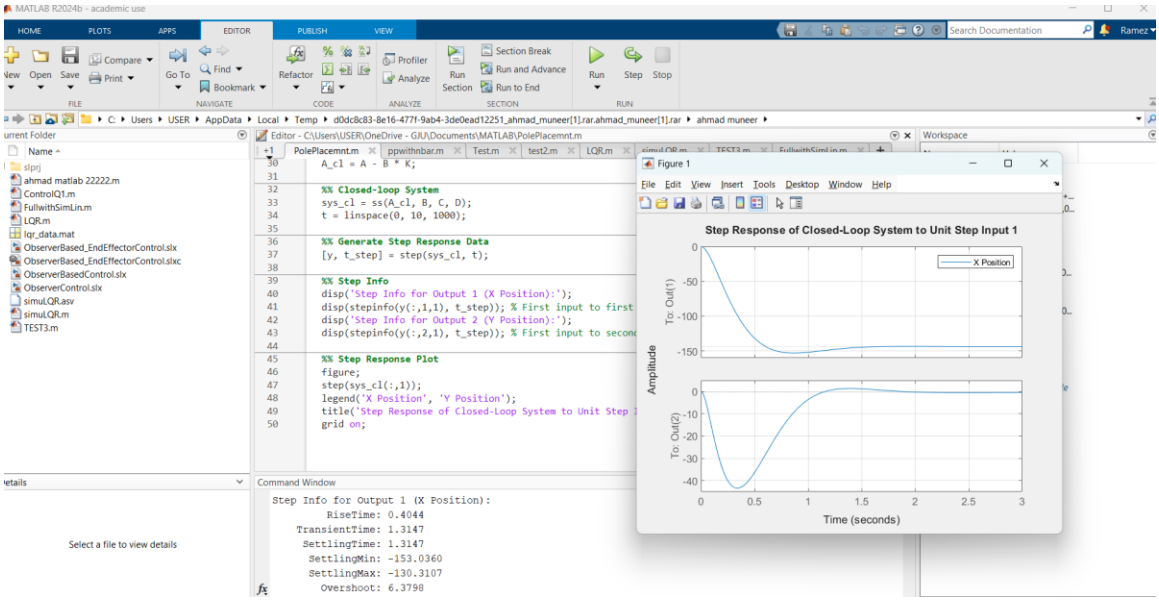


Fig.2

Controller & Observer (test2.m)

In this section we will explain each part of the final MATLAB script including the observer.

Note that this script requires fsolve function to be installed.

1. Inverse Kinematics

a) Motive:

To compute the joint angles (θ0, θ1) that place the end-effector at a desired Cartesian position (Ex, Ey).

b) Implementation:

- fsolve numerically solves nonlinear equations that relate joint angles to end-effector position.
- An initial guess of [0; 0] is provided.
- The equations come from geometry using trigonometry for the 5-bar planar manipulator configuration.

2. Forward Kinematics Check

a) Motive:

To verify that the computed joint angles (θ0, θ1) achieve the target position.

b) Implementation:

- The forward kinematics equations compute Ex_check and Ey_check based on the solved angles.
- These are compared to the desired Ex, Ey.

3. Linearized State-Space Model

a) Motive:

To model the manipulator's motion using a linear approximation around the equilibrium point for control design.

b) Implementation:

- Defines A, B, C, D matrices assuming linearized dynamics.
- A and B describe the system; C maps states to outputs; D is zero (no direct feedthrough).

4. Controllability & Observability

a) Motive:

To verify if the system can be controlled and observed.

b) Implementation:

- **ctrb** and **obsv** generate controllability and observability matrices.
- Their ranks are compared to the number of states to confirm full controllability/observability.

5. Pole Placement Controller

a) Motive:

To design a state feedback controller that stabilizes the system and meets performance specs.

b) Implementation:

- Poles are selected (either fast or slow) to shape the system's response, poles ([-2+1i, -2-1i, -3+0.5i, -3-0.5i]) were chosen through tuning to balance speed and damping.

- Place computers gain K, and the closed-loop system matrix is updated as A - BK.

6. Precompensator (Nbar)

a) Motive:

To eliminate steady-state error in tracking a constant reference input.

b) Implementation:

- Uses the formula Nbar = pinv(C*inv(-A_cl)*B) for full-state feedback systems.
- Ensures correct scaling of input reference to achieve target output.

7. Closed-loop Simulation

a) Motive:

To simulate system response under feedback control without observer estimation.

b) Implementation:

- Uses lsim for simulation with initial condition x0 = [0; 0; 0; 0].
- Input r is a constant desired position.
- Plots compare actual and desired output.

Fig.3 shows the response for the system that meets the requirements.

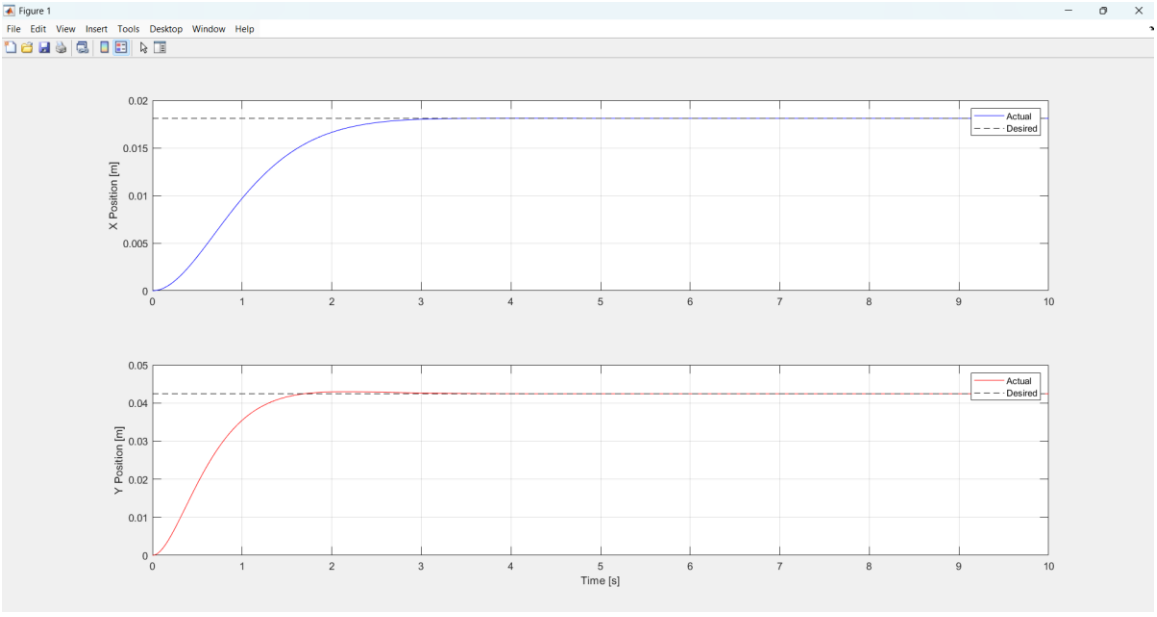


Fig.3

8. Observer Design

a) Motive:

To estimate the full state vector from output measurements for situations where not all States are directly measured.

b) Implementation:

- Places observer poles faster than system poles for rapid convergence.
- Gain matrix L is computed via transposed place.

9. Augmented Observer System

a) Motive:

To simulate combined plant and observer dynamics to assess estimation-based control.

b) Implementation:

- Constructs A_aug, B_aug, C_aug for augmented system.
- Simulates system using lsim with initial observer and plant states.
- Compares outputs to reference.

Fig.4 shows the response for the observer system.

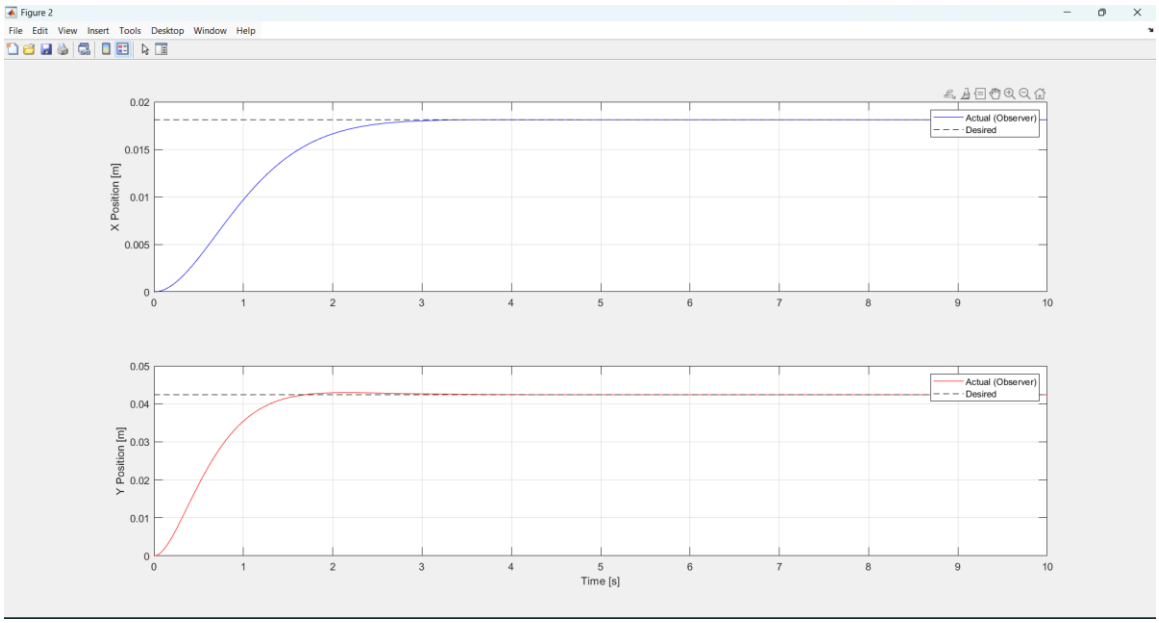


Fig.4

10. Workspace Visualization

a) Motive:

To visualize which positions the manipulator can physically reach, i.e., its workspace.

b) Implementation:

- Uses meshgrid of angle pairs to compute all possible Ex, Ey.
- Plots two versions:
    - Full theoretical range (−π to π), Shows the ideal workspace assuming unlimited joint rotation, without physical constraints.
    - Physically constrained range (−π/2 to π/2), Reflects the actual reachable area, limited by the robot's physical joint limits.
- Desired position is overlaid on both plots.

Fig.5 shows the theoretical and actual range for the workspace.

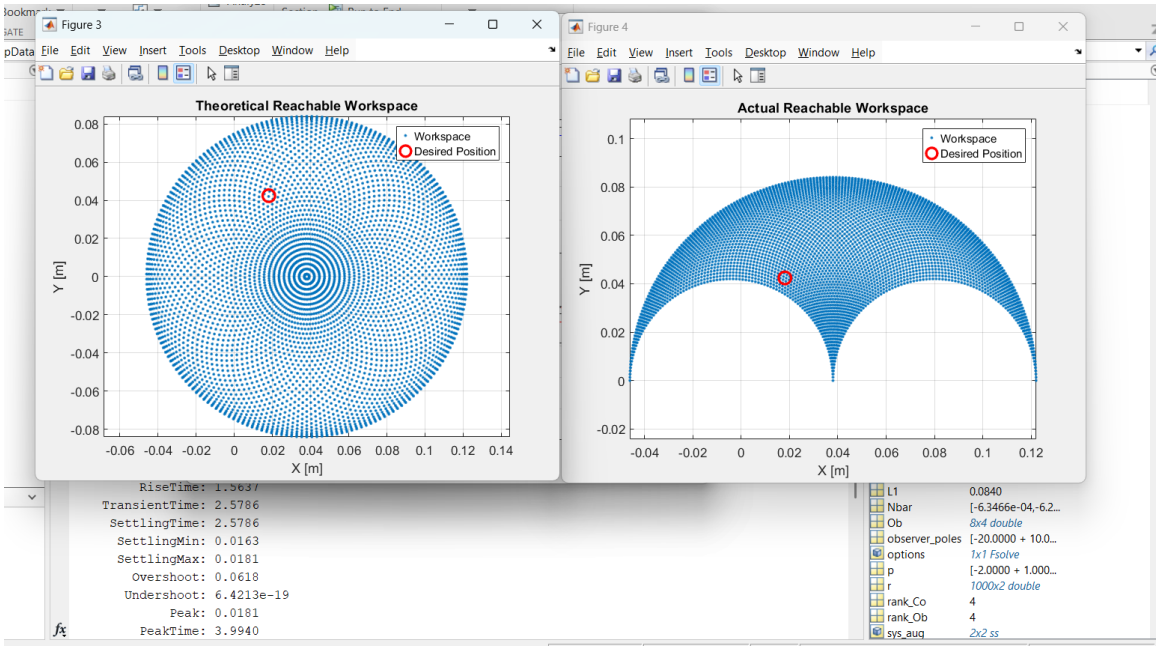


Fig.5

Where the system achieved less than 5% overshoot and less than 3s settling time.

LQR Control Analysis (LQR.m)

Implementation

The LQR controller was implemented to control the end-effector position of the 5-bar planar manipulator directly in task space. The system was linearized around the desired operating point (Ex = 3.8×10^{-2} m, Ey = 8×10^{-2} m).

The implementation includes controllability checks and uses the precompensator **Nbar** for zero steady-state error tracking.

Why it has Failed

The LQR controller failed due to two fundamental issues:

1. **Extreme Pole Placement**

The closed-loop system exhibited poles on the order of 10^4, indicating severe numerical conditioning problems.

2. **Reference Tracking Gain (Nbar) Problems**

The calculation of **Nbar** through the precompensator method becomes numerically problematic. The resulting **Nbar** gains become excessively large, destabilizing the system.

So, these extremely fast poles (high frequency dynamics) would demand excessive control effort from the actuators, potentially causing motor burnout due to rapid switching and high current demands. Additionally, such fast dynamics make observer design challenging, as the observer would need to operate at much higher bandwidths than the plant.

Integral Control (Integral .m)

Motive:

In this part of the project, a state-feedback controller with integral action was implemented to track desired end-effector positions for a planar two-link manipulator. The primary motivation was to test the impact and performance of **pole placement alone** — without relying on steady-state error compensation via an Nbar (precompensator) term. By deliberately excluding Nbar, the integral action within the augmented system becomes the sole mechanism responsible for eliminating steady-state error.

Implementation:

Inverse kinematics was first solved numerically to find the joint angles theta 0 and theta 1 for a given end-effector target (Ex,Ey), which were then used to compute the linearized output matrix C.

The system was augmented by adding integral states to track position errors. Pole placement was applied to design the gain matrices K, choosing poles that balanced stability and response speed.

Simulating the closed-loop system with lsim showed successful reference tracking, confirming that pole placement with integral action alone can achieve zero steady-state error — making this a useful baseline before applying precompensation methods like Nbar. However, an observer was not applicable, and the system exhibit weak performance with some values

AI Utilization

It is not a secret that AI tools such as ChatGPT and Claude AI were used, but their usage has proved how AI can be more stupid than humans. However, their usage was limited to the following:

1. Structuring the code and adding comments for clarity.
2. Generating plots, as our limited MATLAB experience, AI tools were used to create plots for responses and visualizations.
3. Helped us finding the appropriate numerical solution for the invers kinematics equation and checking it from the forward kinematics.
4. Offering alternative methods proved to be unhelpful and time-consuming. Except how to calculate Nbar for MIMO Systemsx.
5. Supporting error debugging and troubleshooting (most frequently used).
6. Report improvements.

Appendix 1: MATLAB Commands Glossary

Command	Category	Description	Example Usage
clc	Environment	Clear command window display	clc;
clear	Environment	Remove all variables from workspace	clear;
optimoptions	Optimization	Create options structure for optimization functions	options = optimoptions('fsolve','Display','off');
fsolve	Optimization	Solve system of nonlinear equations	theta_sol = fsolve(@(theta) [...], theta_init, options);
rad2deg	Math	Convert radians to degrees	theta0_deg = rad2deg(theta0);
fprintf	Display	Print formatted text to command window	fprintf('θ_0 = %.2f deg\n', theta0_deg);
disp	Display	Display variable or text	disp(poles_cl);
zeros	Matrix	Create matrix of zeros	D = zeros(2,2);
diag	Matrix	Create diagonal matrix or extract diagonal	Q = diag([100, 100, 10, 10]);
ctrb	Control Systems	Compute controllability matrix	Co = ctrb(A, B);

obsv	Control Systems	Compute observability matrix	Ob = obsv(A, C);
rank	Linear Algebra	Calculate matrix rank	rank(Co)
size	Matrix	Get matrix dimensions	size(A,1)
lqr	Control Systems	Linear Quadratic Regulator design	K = lqr(A, B, Q, R);
eig	Linear Algebra	Calculate eigenvalues and eigenvectors	poles_cl = eig(Acl);
inv	Linear Algebra	Matrix inverse	Nbar = inv(temp_matrix);
pinv	Linear Algebra	Moore-Penrose pseudoinverse	Nbar = pinv(temp_matrix);
ss	Control Systems	Create state-space model	sys_cl = ss(Acl, B*Nbar, C, D);
linspace	Arrays	Create linearly spaced vector	t = linspace(0, 10, 1000);
repmat	Arrays	Replicate and tile array	u = repmat(r, 1, length(t));
length	Arrays	Number of elements in array	length(t)
lsim	Control Systems	Simulate linear system time response	[y, t, x] = lsim(sys_cl, u', t, x0);
figure	Plotting	Create new figure window	figure;
plot	Plotting	Create 2D line plot	plot(t, y(:,1), 'b', t, y(:,2), 'r--');
legend	Plotting	Add legend to plot	legend('X Position', 'Y Position');
xlabel	Plotting	Label x-axis	xlabel('Time [s]');

ylabel	Plotting	Label y-axis	ylabel('Position [m]');
title	Plotting	Add title to plot	title('Closed-Loop Step Response');
grid	Plotting	Add grid to plot	grid on;
stepinfo	Control Systems	Calculate step response characteristics	info_x = stepinfo(y(:,1), t, Ex);
place	Control Systems	Pole placement design	K = place(A, B, p);
subplot	Plotting	Create subplot in figure	subplot(2,1,1);
meshgrid	Arrays	Create coordinate matrices from vectors	[X, Y] = meshgrid(theta_vals, theta_vals);
sin	Math	Sine function	L1*sin(theta0)
cos	Math	Cosine function	L1*cos(theta0)
ones	Matrix	Create matrix of ones	Ex*ones(size(t))
axis	Plotting	Set axis properties	axis equal;
hold	Plotting	Hold current plot for additional graphics	hold on;

Appendix 2: References

Nise, N. S. (2019). *Control systems engineering* (7th ed.). John Wiley & Sons.

Mutambara, A. G. O. (2018). *Design and analysis of control systems: Driving the fourth industrial revolution* (2nd ed.). CRC Press.

University of Michigan. (n.d.). Control tutorials for MATLAB and Simulink - Introduction: State space. *Control Tutorials for MATLAB and Simulink*. https://ctms.engin.umich.edu/CTMS/index.php?example=Introduction&section=ControlStateSpace#24

https://aleksandarhaber.com/pole-placement-with-integral-control-action-to-eliminate-steady-state-error-state-space-control-design/